

1. What is Reinforcement Learning, and what are its key real-world examples?

Reinforcement Learning (RL) is a computational approach to learning from interaction. An agent learns how to map situations (states) to actions so as to maximize a numerical reward signal. The agent is not told which actions to take; instead, it must discover which actions yield the most reward by trying them. Actions may affect not only the immediate reward but also the next state, and through that, all subsequent rewards.

Key distinguishing features:

- Learning by trial-and-error interaction with an environment
- Delayed and sparse rewards (credit-assignment problem)
- Sequential decision-making under uncertainty
- Goal of maximizing cumulative (long-term) reward

Key real-world examples:

- Game playing (Chess, Go, Atari, StarCraft, Dota 2)
- Robotics (locomotion, manipulation, autonomous navigation)
- Autonomous driving and drone control
- Recommendation systems and personalized content ranking
- Resource management (data-center cooling, inventory control, bidding in auctions)
- Finance (portfolio optimization, algorithmic trading)
- Healthcare (treatment regime optimization, adaptive clinical trials)
- Large language model alignment via Reinforcement Learning from Human Feedback (RLHF)

3. What are the limitations and scope of Reinforcement Learning?

Scope RL is well-suited for sequential decision-making problems in which:

- An agent interacts with an environment over time
- Feedback is given as a scalar reward signal
- The goal is to maximize long-term cumulative reward
- The dynamics may be unknown or difficult to model analytically

Successful domains include games, robotics, recommendation, resource allocation, and preference-based fine-tuning of large models.

Major Limitations

Limitation	Explanation
Sample inefficiency	Most algorithms need millions of interactions.
Exploration–exploitation dilemma	Hard to balance trying new actions vs. using known good ones.
Reward design / specification	Poorly designed rewards lead to reward hacking or undesired behavior.
Credit assignment	Difficulty in determining which actions caused a delayed reward.
Stability & reproducibility	Deep RL training is often sensitive to hyperparameters and random seeds.
Safety & constraints	Standard RL does not inherently respect hard safety constraints.
Sim-to-real gap	Policies trained in simulation often degrade in the real world.
Partial observability & non-stationarity	Real environments frequently violate the Markov assumption or change over time.
High computational cost	Training large neural policies is resource-intensive.
Limited theoretical guarantees	Especially for deep function approximation.

RL is generally **not** the first choice for pure supervised learning tasks or problems that can be solved efficiently with classical optimal control when an accurate model is available.

2. What is the n-Armed Bandit problem, and why is it important in Reinforcement Learning?

The **n-Armed Bandit Problem** is a basic problem in **Reinforcement Learning (RL)** that demonstrates how an agent makes decisions when it does not know which action will give the best reward.

Imagine a **slot machine with n different arms** (levers):

- Each arm represents an **action**.
- The agent chooses one arm at each time step.
- After choosing an arm, the agent receives a **reward**.
- The reward is uncertain because it comes from an unknown probability distribution.
- The agent must learn which arm gives the highest average reward.

Important Terms

1. Action A_t

The arm selected by the agent at time t .

2. Reward R_t

The numerical reward received after selecting an arm.

3. True Action Value $q^*(a)$

The expected reward obtained when action a is selected:

$$q^*(a) = E[R_t | A_t = a]$$

In simple words, it is the **average reward expected from arm a** .

Example

Suppose there are **3 arms**:

Arm Average Reward

Arm 1 2

Arm 2 5

Arm 3 3

The best arm is **Arm 2**, because it has the highest expected reward of **5**.

However, initially the agent **does not know these values**. It must try different arms and learn from the rewards.

Main Challenge: Exploration vs. Exploitation

The agent has two choices:

- **Exploration:** Try different arms to discover which one is better.
- **Exploitation:** Select the arm currently believed to be the best.

For example, if the agent currently thinks Arm 2 is best, it can:

- continue selecting Arm 2 → **exploitation**
- try Arm 1 or Arm 3 to gather more information → **exploration**

Importance in RL

- It isolates the pure **exploration–exploitation dilemma**: the agent must balance pulling the currently best-looking arm (exploitation) versus trying other arms to gather more information (exploration).

- There is only **one state**, so the problem has no sequential state transitions. This makes it the simplest non-trivial setting in which to study value estimation, exploration strategies, and regret.
- Many ideas that later appear in full MDPs (ϵ -greedy, UCB, optimistic initialization, incremental updates, non-stationary tracking) are first developed and understood here.
- It serves as a building block for contextual bandits and as a testbed for exploration methods used in larger RL problems.

3. How is the Agent–Environment Interface defined in Reinforcement Learning?

In Reinforcement Learning, the agent and the environment interact continuously in a loop at discrete time steps $t=0,1,2,\dots$ $t = 0, 1, 2, \dots$:

- At each time step t , the agent receives a representation of the environment's **state**, $S_t \in \mathcal{S}$.
- On the basis of that state, the agent selects an **action**, $A_t \in \mathcal{A}(S_t)$.
- One time step later, as a consequence of its action, the agent receives a numerical **reward**, $R_{t+1} \in \mathbb{R}$, and finds itself in a new state, S_{t+1} .

This interaction is often written as the sequence:

$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$

Key points:

- The **agent** is the learner and decision-maker.
- The **environment** is everything outside the agent (including the reward signal generator).
- The boundary between agent and environment is defined by what the agent can control directly versus what it can only influence indirectly.
- The framework is abstract and flexible: states, actions, and rewards can be defined at whatever level of abstraction is appropriate for the problem.